Robotik Mühendisliği Serisi — Cilt 6

Bacaklı Robot Entegrasyonu

LIPM, ZMP, DCM, yakalanabilirlik, MPC, tüm-gövde kontrolü, öğrenme

Türkçe–İngilizce karma konu anlatımı
teori · sayısal örnekler · çalışan Python ve C++ kodları

Odak: bacaklı ve humanoid robotlar

31 Ekim 2026

İçindekiler

Bu Cilt Hakkında **1**

1 LIPM, ZMP ve DCM **2**

1.1 Doğrusal ters sarkaç 2

1.2 DCM (capture point) 2

2 İtmeden Kurtulma ve Yakalanabilirlik **7**

3 ZMP MPC ve Kendi QP Çözücümüz **10**

3.1 Problem 10

3.2 ADMM ile QP 10

4 Merkezcil Dinamik ve Tüm-Gövde Kontrolü **16**

4.1 Merkezcil dinamik 16

4.2 QP tabanlı tüm-gövde kontrolü (WBC) 16

5 Öğrenme Tabanlı Yürüme **20**

5.1 RL ile yürüme: tarifin parçaları 20

5.2 Taklit ve difüzyon politikaları 20

5.3 Model tabanlı mı, öğrenme mi? 21

5.4 SB-1 için bir RL planı (simülasyonda) 21

5.5 Okuma listesi 21

6 SB-1 Yazılım Mimarisi ve Proje Yol Haritası **22**

6.1 Katmanlar ve hızlar 22

6.2 ROS 2 düğüm taslağı 22

6.3 Proje basamakları (Cilt 0'daki P1–P8 ile uyumlu) 22

7 Sorular **24**

8 Cevaplar **26**

Bu Cilt Hakkında

Cilt 6, serinin önceki tüm parçalarını tek bir hedefte birleştiriyor: **iki bacaklı bir robotu yürütmek ve itildiğinde düşürmemek.** Kinematik ve dinamik (Cilt 1), donanım ve sürücüler (Cilt 3), kontrol (Cilt 4A/4B) ve durum kestirimi (Cilt 5) burada bir araya geliyor. Kullanılan örnek her yerde SB-1: kütle 2.5 kg, kütle merkezi yüksekliği $z_c = 0.32$ m, uyluk ve baldır 0.2 m, ayak tabanı bileğin 3 cm gerisinden 5 cm önüne.

Yöntem sırası, bugünkü humanoidlerin (ve Tedrake'in 5. bölümünün) izlediği yol: doğrusal ters sarkaç (LIPM) → ZMP ve DCM (capture point) → yakalanabilirlik ve adım ayarı → kısıtlı MPC (kendi QP çözücümüzle) → merkezcil dinamik ve QP tabanlı tüm-gövde kontrolü → öğrenme tabanlı yürüme. Son bölüm, SB-1 için tüm bunları bir yazılım mimarisine bağlıyor.

Önkoşullar: Cilt 1 Bölüm 4–6 (Jacobian, statik, dinamik), Cilt 4A, Cilt 4B Bölüm 4, 5, 10, 11. **Staj dönemi için:** telefon sürümü (Kitapçık 6) kodsuz, 20 soru-cevaplı.

Bölüm	Konu	Kod
1	LIPM, ZMP, DCM; DCM ile adım planı ve izleme	`w1_lipm_dcm.py`, `w1_dcm.cpp`
2	İtmeden kurtulma, N-adım yakalanabilirlik	`w2_capture.py`
3	ZMP MPC ve kendi QP çözücümüz (ADMM)	`w3_mpc.py`, `qp_admm.py`, `w3_admm.cpp`
4	Merkezcil dinamik, kuvvet dağılımı, sürtünme konisi, tork; SB-1 için tasarım sonucu	`w4_wbc.py`
5	Öğrenme tabanlı yürüme: RL, sim-to-real, taklit ve difüzyon politikaları	–
6	SB-1 yazılım mimarisi ve proje yol haritası	–
7–8	Sorular ve cevaplar	–

Bu ciltteki modeller sadeleştirilmiş

Hesaplar sagittal düzlemde (öne-arkaya) yapıldı; yan salınım (lateral) aynı denklemlerle ayrı bir eksende çözülür. Çift destek fazı, ayak salınım yörüngesi ve bacak kütlesi ihmal edildi. Amaç fikirleri temiz göstermek; gerçek robotta bu ihmallerin her biri birkaç santimetrelik hata demektir ve geri besleme ile kapatılır.

Bölüm 1

LIPM, ZMP ve DCM

1.1 Doğrusal ters sarkaç

Bütün kütle gövdede bir noktada, yükseklik sabit ($z = z_c$), bacaklar kütlesiz ve kütle merkezi etrafında açısal momentum değişmiyor kabul edilirse, yatay dinamik doğrusaldır:

$$\ddot{x} = \omega^2 (x - p), \qquad \omega = \sqrt{g/z_c}.$$

p **ZMP** (sıfır moment noktası): yer tepki kuvvetinin etki noktası. ZMP sadece *destek poligonu* (yerdeki ayakların dışbükey zarfı) içinde olabilir; dışarı "gitmesi" istenirse robot ayak kenarı etrafında devrilmeye başlar. Kontrolün tüm hikâyesi budur: CoM'u, ZMP'yi destek içinde tutarak götürmek.

p sabitken çözüm $x(t) - p = (x_0 - p)\cosh \omega t + \frac{\dot{x}_0}{\omega} \sinh \omega t$: **üstel olarak kaçar**. SB-1'de $\omega = 5.54$ 1/s, zaman sabiti 181 ms: ayağın 2 cm önündeki CoM, ayak altında hiçbir şey yapılmazsa 0.4 s'de 9.3 cm'ye kaçar.

1.2 DCM (capture point)

$\xi = x + \dot{x}/\omega$ değişkeniyle sistem ikiye ayrılır:

$$\dot{\xi} = \omega(\xi - p) \quad \text{(kararsız)}, \qquad \dot{x} = -\omega(x - \xi) \quad \text{(kararlı)}.$$

CoM, DCM'yi kendiliğinden izler; kontrol edilmesi gereken *tek* şey DCM'dir. $p = \xi$ yapılırsa DCM durur ve CoM ona yaklaşıp durur: "capture point" adı buradan (ayağı oraya basarsan durursun).

DCM ile adım planı. Ayak konumları $p_0, p_1, \ldots$, adım süresi T. Son ayakta durulacağı için son DCM son ayağın üstünde. Her adım sırasında ZMP o adımın ayağında sabitse $\xi(t) = p_i + e^{\omega(t-T)}(\xi_{\text{son},i} - p_i)$; geriye doğru:

$$\xi_{\text{son},i-1} = \xi_{\text{baş},i} = p_i + e^{-\omega T}(\xi_{\text{son},i} - p_i).$$

İzleme kontrolcüsü: istenen $\dot{\xi} = \dot{\xi}_{\text{ref}} - k(\xi - \xi_{\text{ref}})$, bunu üreten ZMP $p = \xi - \dot{\xi}/\omega$; ayak tabanına kırpılır.

Sayılar: SB-1, 8 cm adım, 0.4 s, $e^{-\omega T} = 0.109$

Adım sonu DCM'leri: 8.98, 16.98, 24.97, 32.87, 40, 40 cm. Elle: $\xi_{\text{son},3} = 32 + 0.109 \cdot (40 - 32) = 32.87$. Bozucu yokken DCM hatası 0.3–0.6 mm. $t = 0.9$ s'de ani hız itmesi ($k = 3$):

itme	max DCM hatası	sonuç
0.10 m/s	1.79 cm	toparladı, ZMP hiç doymadı
0.20 m/s	3.60 cm	toparladı, ZMP %1.4 doyumda
0.30 m/s	–	DCM ıraksadı (1.36 s'de 10 cm): **düştü**

İtme DCM'yi aniden $\Delta v/\omega$ kadar sıçratır (0.3 m/s → 5.4 cm). Ayak tabanı DCM'yi geri çekecek

kadar uzun değilse, **adım yerini değiştirmek** gerekir (Bölüm 2). C++ sürümü aynı sonuçları verdi.

Python

```python
import numpy as np

# Doğrusal ters sarkaç (LIPM), SB-1 ölçüleri: kütle merkezi yüksekliği z_c = 0.32 m
g, zc = 9.81, 0.32
w = np.sqrt(g / zc)                          # LIPM doğal frekansı
print(f"ω = sqrt(g/z_c) = {w:.3f} 1/s, zaman sabiti 1/ω = {1000 / w:.0f} ms")

# 1) Sabit ZMP (p =0) ile analitik çözüm: x(t) = x0 cosh(ωt) + (v0/ω) sinh(ωt)
x0, v0 = 0.02, 0.0
for t in [0.1, 0.2, 0.4]:
    print(f"  x0 =2 cm, ayak altında beklerse t ={t:.1f} s: x = {100 * (x0 * np.cosh(w * t) + v0 / w *
    np.sinh(w * t)):.2f} cm")

# 2) DCM (capture point) ξ = x + ẋ/ω : ξ̇ = ω (ξ - p)  -> kararsız tek mod; x ise ξ'yi kararlı biçimde
izler: ẋ = -ω (x - ξ)
# Adım planı: 8 cm adım, 0.4 s, ileri yürüme (yan salınımı sadeleştirmek için sagittal düzlem)
T, L, nstep = 0.4, 0.08, 6
feet = np.array([0.0] + [L * i for i in range(1, nstep)] + [L * (nstep - 1)])   # son adım ayakları yan
yana: dur
# Geriye doğru özyineleme: her adımın sonunda DCM bir sonraki ayağa göre hesaplanır, son DCM son ayağın
üstünde
xi_eos = np.zeros(len(feet)); xi_eos[-1] = feet[-1]
for i in range(len(feet) - 2, -1, -1):
    xi_eos[i] = feet[i + 1] + np.exp(-w * T) * (xi_eos[i + 1] - feet[i + 1])
xi_ini = np.r_[feet[0] + np.exp(-w * T) * (xi_eos[0] - feet[0]), xi_eos[:-1]]
print("adım başı DCM [cm]:", (100 * xi_ini).round(2))
print("adım sonu DCM [cm]:", (100 * xi_eos).round(2))

def xi_ref(t):
    i = min(int(t / T), len(feet) - 1); tau = t - i * T
    return feet[i] + np.exp(w * (tau - T)) * (xi_eos[i] - feet[i]), feet[i]

# 3) DCM izleme kontrolcüsü ve simülasyon
foot_back, foot_front = -0.03, 0.05        # ayak bileğine göre ayak tabanı sınırları
def simulate(k, push_t=None, push_dv=0.0, dt=1e-3, tend=None):
    tend = tend or len(feet) * T
    xi_r0 = xi_ini[0]
    x, v = xi_r0, 0.0                       # başlangıçta CoM, DCM ile aynı yerde ve duruyor (ξ = x)
    log = []
    for kk in range(int(tend / dt)):
        t = kk * dt
        if push_t is not None and abs(t - push_t) < dt / 2: v += push_dv
        xr, foot = xi_ref(t); xr_dot = w * (xr - foot)
        xi = x + v / w
        p = xi - (xr_dot - k * (xi - xr)) / w           # istenen DCM hızını üreten ZMP
        p_cl = np.clip(p, foot + foot_back, foot + foot_front)   # ZMP ayak tabanını terk edemez!
        a = w**2 * (x - p_cl); v += a * dt; x += v * dt
        log.append((t, x, xi, xr, p, p_cl, foot))
    return np.array(log)

for k in [0.0, 1.0, 3.0]:
    lg = simulate(k)
    print(f"k ={k:.0f}: max |ξ-ξ_ref| = {1000 * np.abs(lg[:, 2] - lg[:, 3]).max():.2f} mm, "
```

```
          f"ZMP ayaktan max uzaklık: {100 * np.abs(lg[:, 5] - lg[:, 6]).max():.2f} cm, son CoM {100 *
          lg[-1, 1]:.2f} cm")
for dv in [0.10, 0.20, 0.30]:
    lg = simulate(3.0, push_t=0.9, push_dv=dv)
    sat = (np.abs(lg[:, 4] - lg[:, 5]) > 1e-9).mean() * 100
    err = np.abs(lg[:, 2] - lg[:, 3])
    if err[-1] > 0.02:
        div = lg[np.argmax(err > 0.10), 0]
        print(f"t =0.9 s'de {dv:.2f} m/s itme: DCM ıraksadı (hata t ={div:.2f} s'de 10 cm'yi geçti) ->
        DÜŞTÜ; adım yeri değişmeliydi")
    else:
        print(f"t =0.9 s'de {dv:.2f} m/s itme: max DCM hatası {100 * err.max():.2f} cm, son hata {100 *
        err[-1]:.2f} cm, ZMP doyumda %{sat:.1f}")
print(f"ayak bileği stratejisi sınırı (DCM sıçraması v/ω ≤ ayak önü): v ≤ {foot_front * w:.3f} m/s")
```

Çıktı:

```
ω = sqrt(g/z_c) = 5.537 1/s, zaman sabiti 1/ω = 181 ms
  x0 =2 cm, ayak altında beklerse t =0.1 s: x = 2.31 cm
  x0 =2 cm, ayak altında beklerse t =0.2 s: x = 3.36 cm
  x0 =2 cm, ayak altında beklerse t =0.4 s: x = 9.27 cm
adım başı DCM [cm]: [ 0.98  8.98 16.98 24.97 32.87 40.   40.  ]
adım sonu DCM [cm]: [ 8.98 16.98 24.97 32.87 40.   40.   40.  ]
k =0: max |ξ-ξ_ref| = 0.63 mm, ZMP ayaktan max uzaklık: 0.06 cm, son CoM 39.88 cm
k =1: max |ξ-ξ_ref| = 0.32 mm, ZMP ayaktan max uzaklık: 0.04 cm, son CoM 39.92 cm
k =3: max |ξ-ξ_ref| = 0.32 mm, ZMP ayaktan max uzaklık: 0.05 cm, son CoM 39.94 cm
t =0.9 s'de 0.10 m/s itme: max DCM hatası 1.79 cm, son hata 0.00 cm, ZMP doyumda %0.0
t =0.9 s'de 0.20 m/s itme: max DCM hatası 3.60 cm, son hata 0.01 cm, ZMP doyumda %1.4
t =0.9 s'de 0.30 m/s itme: DCM ıraksadı (hata t =1.36 s'de 10 cm'yi geçti) -> DÜŞTÜ; adım yeri değişmeliydi
ayak bileği stratejisi sınırı (DCM sıçraması v/ω ≤ ayak önü): v ≤ 0.277 m/s
```

C++: gerçek zamanlı döngüye uygun DCM planlayıcı ve kontrolcü

```cpp
// w1_dcm.cpp — DCM adım planı (geriye özyineleme) ve DCM izleme kontrolcüsü; 1 kHz gerçek zamanlı
döngüye
// konacak biçimde: her çağrıda O(1) iş, bellek ayırma yok.
#include <algorithm>
#include <array>
#include <cmath>
#include <cstdio>

constexpr double kG = 9.81, kZc = 0.32;
const double kW = std::sqrt(kG / kZc);

template <int NF>
struct DcmPlan {
    std::array<double, NF> feet{}, xi_eos{};
    double T;
    DcmPlan(const std::array<double, NF>& f, double step_time) : feet(f), T(step_time) {
        xi_eos[NF - 1] = feet[NF - 1];                                  // son: ayağın üstünde dur
        for (int i = NF - 2; i >= 0; --i)
            xi_eos[i] = feet[i + 1] + std::exp(-kW * T) * (xi_eos[i + 1] - feet[i + 1]);
    }
    double xi_ini0() const { return feet[0] + std::exp(-kW * T) * (xi_eos[0] - feet[0]); }
    // t anındaki referans DCM ve o anki destek ayağı
    void ref(double t, double& xi, double& foot) const {
        const int i = std::min(int(t / T), NF - 1);
        foot = feet[i];
        xi = foot + std::exp(kW * (t - i * T - T)) * (xi_eos[i] - foot);
    }
```

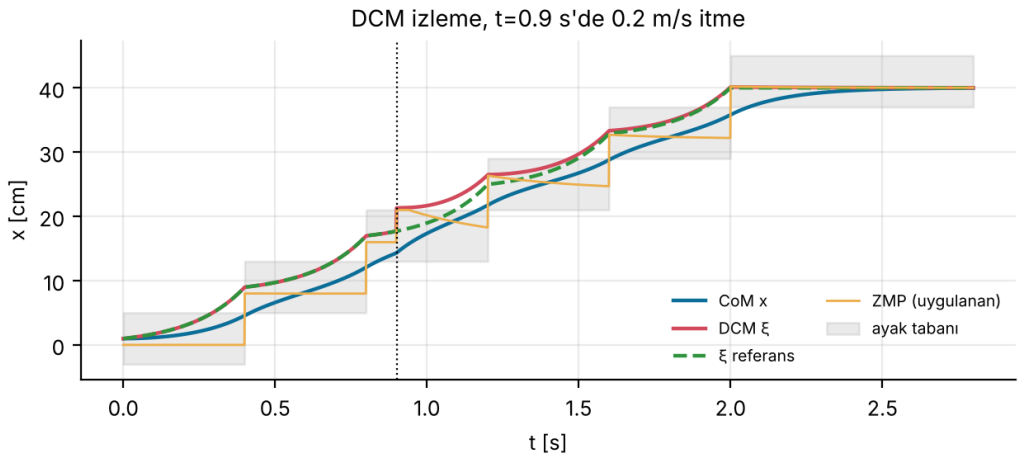
```cpp
};

class DcmController {
public:
    DcmController(double k, double back, double front) : k_(k), back_(back), front_(front) {}
    // ölçülen CoM konumu/hızı -> uygulanacak ZMP (ayak tabanına kırpılmış)
    double zmp(double x, double v, double xi_r, double foot) const {
        const double xi = x + v / kW, xi_r_dot = kW * (xi_r - foot);
        const double p = xi - (xi_r_dot - k_ * (xi - xi_r)) / kW;
        return std::clamp(p, foot + back_, foot + front_);
    }
private:
    double k_, back_, front_;
};

int main() {
    const DcmPlan<7> plan({0.0, 0.08, 0.16, 0.24, 0.32, 0.40, 0.40}, 0.4);
    std::printf("ω = %.3f 1/s\nadım sonu DCM [cm]:", kW);
    for (double v : plan.xi_eos) std::printf(" %.2f", 100 * v);
    std::printf("\n");
    const DcmController ctl(3.0, -0.03, 0.05);
    for (double dv : {0.0, 0.10, 0.20, 0.30}) {
        double x = plan.xi_ini0(), v = 0, err_max = 0, err = 0; const double dt = 1e-3;
        for (int k = 0; k < int(7 * 0.4 / dt); ++k) {
            const double t = k * dt;
            if (std::abs(t - 0.9) < dt / 2) v += dv;
            double xr, foot; plan.ref(t, xr, foot);
            const double p = ctl.zmp(x, v, xr, foot);
            v += kW * kW * (x - p) * dt; x += v * dt;
            err = std::abs(x + v / kW - xr); err_max = std::max(err_max, err);
        }
        if (err > 0.02) std::printf("itme %.2f m/s: DCM ıraksadı -> DÜŞTÜ\n", dv);
        else std::printf("itme %.2f m/s: max DCM hatası %.2f cm, son CoM %.2f cm\n", dv, 100 * err_max,
100 * x);
    }
    return 0;
}
```

Çıktı:

```
ω = 5.537 1/s
adım sonu DCM [cm]: 8.98 16.98 24.97 32.87 40.00 40.00 40.00
itme 0.00 m/s: max DCM hatası 0.05 cm, son CoM 39.94 cm
itme 0.10 m/s: max DCM hatası 1.79 cm, son CoM 39.95 cm
itme 0.20 m/s: max DCM hatası 3.60 cm, son CoM 39.96 cm
itme 0.30 m/s: DCM ıraksadı -> DÜŞTÜ
```



Şekil 1.1: DCM izleme, $t = 0.9$ s'de 0.2 m/s itme. Gri bantlar ayak tabanı: ZMP (kırmızı) her an içinde kalmak zorunda.

C++ notu: `std::array` ve `constexpr`

`template <int NF> struct DcmPlan` içindeki `std::array<double, NF>` boyutu derleme zamanında bilinen, yığında duran dizi: gerçek zamanlı döngüde bellek ayırma yok. `constexpr double kG = 9.81` derleme zamanı sabiti. `std::clamp(p, lo, hi)` (C++17) kırpma.

Bölüm 2

İtmeden Kurtulma ve Yakalanabilirlik

Bir itmeden sonra robot "yakalanabilir mi" (durabilir mi)? Üç strateji, artan güçte:

1. **Ayak bileği:** ZMP'yi ayak tabanı içinde kaydır. DCM, ayağın ön kenarını geçmediyse yeterli: $\xi_0 = v/\omega \le$ ön $\Rightarrow v \le \omega \cdot$ ön.
2. **Kalça (açısal momentum):** gövdeyi döndürerek kısa süreli ek moment (LIPM'in dışında; burada hesaplanmadı).
3. **Adım atma:** ayağı DCM'nin yakınına bas. Salınım süresi T_s boyunca DCM kaçmaya devam eder: $\xi(T_s) = p + (\xi_0 - p)e^{\omega T_s}$.

N-adım yakalanabilirlik (Koolen, Pratt ve ark.): "en fazla N adımda durabileceğim durumlar kümesi". Bacak erimi r, ayak önü a ile geriye doğru özyineleme:

$$\ell_0 = a, \qquad \ell_n = a + (r + \ell_{n-1} - a)\, e^{-\omega T_s}, \qquad v_{\max,n} = \omega\, \ell_n.$$

Sayılar: SB-1, ayak önü 5 cm, erim 20 cm

Ayak bileği: $v \le 5.537 \cdot 0.05 = 0.277$ m/s.

salınım T_s	$e^{-\omega T_s}$	1 adım	2 adım	3 adım
0.15 s	0.436	0.759	0.970	1.061
0.20 s	0.330	0.643	0.764	0.804
0.30 s	0.190	0.487	0.527	0.535
0.40 s	0.109	0.398	0.411	0.412
0.60 s	0.036	0.317	0.318	0.318

Hızlı adım, uzak bacaktan daha değerli: 0.6 s'lik yavaş bir adım, ayak bileğinin üzerine çok az ekliyor. Benzetim ($T_s = 0.25$ s; teori 1 adım için 0.554 m/s): 0.4 m/s itmede sabit 10 cm'lik adım bile yetti; 0.5 m/s'de sabit adım **düştü**, DCM'ye basan adım (20 cm, erimin sınırında) **durdu**; 0.6 m/s'de ikisi de yetmedi (ikinci adım gerekir).

SB-1 için anlamı: MG996R'nin hızı (6 V'ta 0.14 s/60°) ve 20 ms komut gecikmesi (Cilt 3) ile 0.2 s'lik bir salınım iddialı; SB-1'in gerçekçi itme toleransı ayak bileği sınırına (0.28 m/s) yakın olacaktır. Bunu masaüstü bacakla ölçmek güzel bir deney.

Python

```python
import numpy as np

# İtmeden kurtulma: ayak bileği stratejisi mi, adım atma mı?  (SB-1: z_c = 0.32 m, ayak önü 5 cm, bacak erimi 20 cm)
g, zc = 9.81, 0.32; w = np.sqrt(g / zc)
front, reach = 0.05, 0.20

# Robot ayakta duruyor (x = 0, ayak 0'da). İtme sonrası hız v -> DCM ξ0 = v/ω.
# Ayak bileği: ZMP en fazla 'front' ileri gidebilir. ξ0 ≤ front ise yerinde durabilir.
```

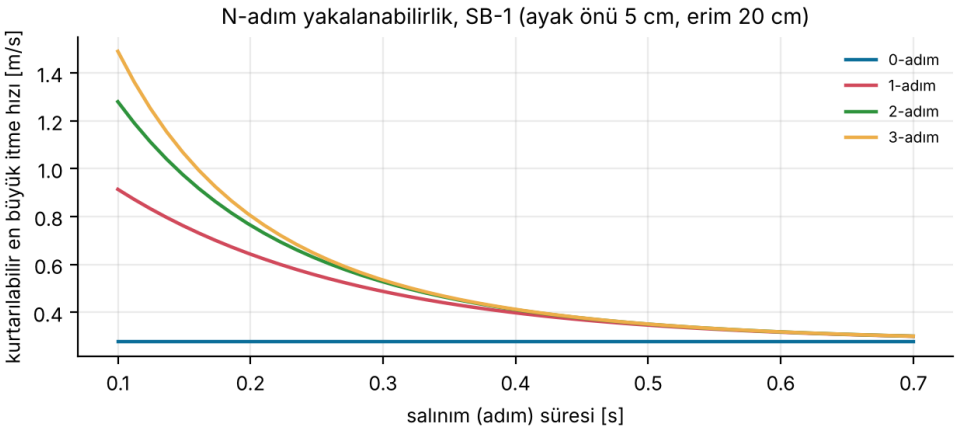
```python
print(f"0-adım (ayak bileği) sınırı: v ≤ ω·front = {w * front:.3f} m/s")

# N-adım: her adımda ayak en fazla 'reach' öne konabilir; salınım süresi Ts boyunca ZMP eski ayağın ön kenarında.
# Son ayakta durabilmek için DCM, ayağın en fazla 'front' önünde olmalı. Geriye doğru:
#   ξ(adım anı) - eski ayak ≤ reach + lim  ve  ξ(Ts) = front + (ξ0 - front) e^{ω Ts}
def n_step_vmax(n, Ts):
    lim = front                                              # 0 adım: ayak bileği
    for _ in range(n):
        lim = front + (reach + lim - front) * np.exp(-w * Ts)    # bir adım daha atılabilirse izin verilen ξ0
    return w * lim
for Ts in [0.15, 0.2, 0.3, 0.4, 0.6]:
    print(f"salınım {Ts:.2f} s (e^(-ωTs) ={np.exp(-w * Ts):.3f}): " + ", ".join(f"N ={n}: {n_step_vmax(n, Ts):.3f}" for n in range(0, 4)) + " m/s")

# Simülasyon: itme, ayak 0.25 s sonra ya DCM'ye ya da sabit 10 cm öne konuyor
def push_sim(v0, Ts, step_rule="dcm", dt=1e-4, T=2.0):
    x, v = 0.0, v0; foot = 0.0; stepped = False; log = []
    for k in range(int(T / dt)):
        t = k * dt; xi = x + v / w
        if not stepped and t >= Ts:
           target = xi if step_rule == "dcm" else foot + 0.10       # DCM'ye bas ya da sabit 10 cm adım
            foot = foot + np.clip(target - foot, -reach, reach); stepped = True
        # adımdan önce ZMP eski ayağın ön kenarında; sonra ξ̇ = -ω(ξ - ayak) isteyen ZMP: p = ξ + (ξ - ayak)
        p = np.clip(2 * xi - foot, foot - 0.03, foot + front) if stepped else foot + front
        a = w**2 * (x - p); v += a * dt; x += v * dt
        log.append((t, x, xi, p, foot))
    return np.array(log)
for v0, rule in [(0.4, "sabit"), (0.5, "sabit"), (0.5, "dcm"), (0.6, "dcm")]:
    lg = push_sim(v0, 0.25, rule)
    final_xi_err = lg[-1, 2] - lg[-1, 4]
    ok = abs(final_xi_err) < 0.01
    print(f"itme {v0} m/s, adım kuralı '{rule}': ayak {100 * lg[-1, 4]:.1f} cm'ye kondu -> "
          + (f"DURDU (CoM {100 * lg[-1, 1]:.1f} cm)" if ok else "DÜŞTÜ (DCM ayaktan kaçtı)"))
print(f"teori, salınım 0.25 s, 1 adım: v ≤ {n_step_vmax(1, 0.25):.3f} m/s")
```

Çıktı:

```
0-adım (ayak bileği) sınırı: v ≤ ω·front = 0.277 m/s
salınım 0.15 s (e^(-ωTs) =0.436): N =0: 0.277, N =1: 0.759, N =2: 0.970, N =3: 1.061 m/s
salınım 0.20 s (e^(-ωTs) =0.330): N =0: 0.277, N =1: 0.643, N =2: 0.764, N =3: 0.804 m/s
salınım 0.30 s (e^(-ωTs) =0.190): N =0: 0.277, N =1: 0.487, N =2: 0.527, N =3: 0.535 m/s
salınım 0.40 s (e^(-ωTs) =0.109): N =0: 0.277, N =1: 0.398, N =2: 0.411, N =3: 0.412 m/s
salınım 0.60 s (e^(-ωTs) =0.036): N =0: 0.277, N =1: 0.317, N =2: 0.318, N =3: 0.318 m/s
itme 0.4 m/s, adım kuralı 'sabit': ayak 10.0 cm'ye kondu -> DURDU (CoM 10.0 cm)
itme 0.5 m/s, adım kuralı 'sabit': ayak 10.0 cm'ye kondu -> DÜŞTÜ (DCM ayaktan kaçtı)
itme 0.5 m/s, adım kuralı 'dcm': ayak 20.0 cm'ye kondu -> DURDU (CoM 20.0 cm)
itme 0.6 m/s, adım kuralı 'dcm': ayak 20.0 cm'ye kondu -> DÜŞTÜ (DCM ayaktan kaçtı)
teori, salınım 0.25 s, 1 adım: v ≤ 0.554 m/s
```



Şekil 2.1: Kurtarılabilir en büyük itme hızı, salınım süresi ve adım sayısına göre.

Bölüm 3

ZMP MPC ve Kendi QP Çözücümüz

3.1 Problem

DCM kontrolcüsü ZMP'yi "son anda" kırpıyor; kısıtı önceden bilmiyor. **Model öngörülü kontrol** (Kajita'nın önizleme kontrolü, Wieber'in MPC'si) ufuk boyunca ZMP'nin ayak içinde kalmasını *planın parçası* yapar. Durum $\mathbf{s} = (x, \dot{x}, \ddot{x})$, giriş jerk j, $\Delta t = 50$ ms:

$$\mathbf{s}_{k+1} = \mathbf{A}\mathbf{s}_k + \mathbf{B}j_k, \qquad z_k = \mathbf{C}\mathbf{s}_k = x_k - \ddot{x}_k/\omega^2.$$

Ufuk $N = 32$ (1.6 s) boyunca ZMP'ler doğrusal: $\mathbf{Z} = \mathbf{P}_x\mathbf{s}_0 + \mathbf{P}_u\mathbf{J}$. QP:

$$\min_{\mathbf{J}} \ \tfrac{a}{2}\|\mathbf{J}\|^2 + \tfrac{\gamma}{2}\|\mathbf{Z} - \mathbf{Z}_{\text{ref}}\|^2 \quad \text{s.t.} \quad \mathbf{z}_{\min} \le \mathbf{P}_x\mathbf{s}_0 + \mathbf{P}_u\mathbf{J} \le \mathbf{z}_{\max}.$$

Her 50 ms'de çöz, sadece ilk girişi uygula, tekrarla (kayan ufuk; Cilt 4B Bölüm 11).

3.2 ADMM ile QP

OSQP'nin algoritması: $\min \frac{1}{2}\mathbf{x}^T\mathbf{P}\mathbf{x} + \mathbf{q}^T\mathbf{x}$, $\mathbf{l} \le \mathbf{A}\mathbf{x} \le \mathbf{u}$. Yardımcı $\mathbf{z} = \mathbf{A}\mathbf{x}$ ve ikil (dual) $\mathbf{y}$ ile:

1. $(\mathbf{P} + \sigma\mathbf{I} + \rho\mathbf{A}^T\mathbf{A})\tilde{\mathbf{x}} = \sigma\mathbf{x} - \mathbf{q} + \mathbf{A}^T(\rho\mathbf{z} - \mathbf{y})$ (matris sabit: bir kez Cholesky),
2. $\mathbf{z} \leftarrow \Pi_{[\mathbf{l},\mathbf{u}]}(\alpha\mathbf{A}\tilde{\mathbf{x}} + (1-\alpha)\mathbf{z} + \mathbf{y}/\rho)$ (izdüşüm = kırpma),
3. $\mathbf{y} \leftarrow \mathbf{y} + \rho(\cdot)$; birincil ve ikil artık küçülünce dur.

Her iterasyon iki üçgen çözüm ve bir kırpma: çok ucuz, gömülü sistemlere uygun, **sıcak başlangıçla** (önceki çözüm) MPC'de hızlı. Olurlu olmayan (infeasible) problemde yakınsamaz; bu da bir bilgi: "bu adım planıyla kurtulamazsın".

Sayılar: 0.4 s'de 8 cm adım, $t = 0.9$ s'de itme

itme	kısıtsız: ZMP komutu ayak dışına	kısıtlı (ADMM): ayak dışına	ADMM iterasyon (ort./max)
0	0	0	19 / 30
0.20 m/s	0	0	21 / 100
0.30 m/s	**3.97 cm**	1.77 cm	665 / 4000 (bazı anlarda olurlu değil)
0.35 m/s	düştü	düştü	–

Dürüst gözlem: ZMP'yi ayak ortasına çeken bir maliyetle kısıtsız MPC de küçük itmelerde aynı sonucu veriyor; kısıtlar **ancak sınıra yaklaşınca** fark yaratıyor. 0.30 m/s'de kısıtsız plan fiziksel olarak imkânsız ZMP'ler (ayağın 4 cm dışı) istiyor; kısıtlı plan isteklerini olabildiğince içeride tutuyor. 0.35 m/s'de *sabit adım planıyla* hiçbir ZMP dizisi yetmiyor: çözüm adım yerini de değişken yapmak (Herdt ve ark. 2010; Bölüm 2).
Doğrulama: tek bir QP'de ADMM amaç değeri -0.01009296, SciPy SLSQP -0.01009296,

ZMP farkı 0.008 mm. C++ sürümü Python ile basamak basamak aynı (19/30 ve 665/4000 iterasyon); 60 MPC adımı yaklaşık 2 ms (itmesiz) ve 65–75 ms (itmeli, olurlu olmayan anlar dahil; süre makineye göre değişir).

Python: QP çözücü (code/lib/qp_admm.py)

```python
"""qp_admm.py — kendi QP çözücümüz (ADMM, OSQP'nin temel algoritması). Cilt 6 Bölüm 3 ve 4."""
import numpy as np

#   min ½ xᵀPx + qᵀx   s.t.  l ≤ A x ≤ u
class ADMM:
    def __init__(self, P, A, rho=0.1, sigma=1e-6, alpha=1.6):
        self.P, self.A, self.rho, self.sigma, self.alpha = P, A, rho, sigma, alpha
        K = P + sigma * np.eye(P.shape[0]) + rho * A.T @ A
        self.L = np.linalg.cholesky(K)                      # bir kez ayrıştır, her iterasyonda iki üçgen çözüm
        self.x = np.zeros(P.shape[0]); self.z = np.zeros(A.shape[0]); self.y = np.zeros(A.shape[0])
    def solve(self, q, l, u, eps=1e-5, max_iter=4000):
        P, A, r, s, a = self.P, self.A, self.rho, self.sigma, self.alpha
        x, z, y = self.x, np.clip(self.z, l, u), self.y      # sıcak başlangıç (warm start)
        for it in range(1, max_iter + 1):
            rhs = s * x - q + A.T @ (r * z - y)
            xt = np.linalg.solve(self.L.T, np.linalg.solve(self.L, rhs))
            zt = A @ xt
            x = a * xt + (1 - a) * x
            zr = a * zt + (1 - a) * z
            z = np.clip(zr + y / r, l, u)                   # kısıt kümesine izdüşüm: sadece kırpma!
            y = y + r * (zr - z)
            if it % 10 == 0:
                rp = np.abs(A @ x - z).max(); rd = np.abs(P @ x + q + A.T @ y).max()
                if rp < eps and rd < eps * 10: break
        self.x, self.z, self.y = x, z, y
        return x, it
```

Python: MPC

```python
import numpy as np

from qp_admm import ADMM                                    # code/lib/qp_admm.py (aşağıda listelendi)

# ---------- LIPM, jerk girişli ayrık model ----------
g, zc = 9.81, 0.32; w2 = g / zc
dt, N = 0.05, 32                                            # 50 ms adım, 1.6 s ufuk
Ad = np.array([[1, dt, dt**2 / 2], [0, 1, dt], [0, 0, 1]]); Bd = np.array([dt**3 / 6, dt**2 / 2, dt])
C = np.array([1, 0, -1 / w2])                               # ZMP = x - ẍ/ω²
Px = np.zeros((N, 3)); Pu = np.zeros((N, N))                # Z = Px s0 + Pu J  (ufuk boyunca ZMP)
Ak = np.eye(3)
for i in range(N):
    Ak = Ad @ Ak; Px[i] = C @ Ak
    for j in range(i + 1):
        Pu[i, j] = C @ np.linalg.matrix_power(Ad, i - j) @ Bd
a_j, g_z = 1e-6, 1.0                                        # jerk ağırlığı, ZMP takip ağırlığı
H = a_j * np.eye(N) + g_z * Pu.T @ Pu

# Adım planı (w1 ile aynı): 0.4 s'de bir 8 cm, ayak tabanı bileğe göre [-3, +5] cm
T_step, L_step = 0.4, 0.08
def foot_at(t): return L_step * min(int(t / T_step), 5)
back, front = -0.03, 0.05
```

```

def run(constrained, push_dv=0.0, push_t=0.9, Tsim=3.0):
    s = np.zeros(3); s[0] = 0.0
    solver = ADMM(H, Pu) if constrained else None
    t = 0.0; log = []; iters = []
    while t < Tsim - 1e-9:
        tk = t + dt * np.arange(1, N + 1)
        f = np.array([foot_at(tt - dt / 2) for tt in tk])      # örnek tt'deki ZMP, (tt-dt, tt] aralığının ayağında
        zref = f + 0.01                                        # ayak tabanının ortası
        q = g_z * Pu.T @ (Px @ s - zref)
        if constrained:
            Jopt, it = solver.solve(q, f + back - Px @ s, f + front - Px @ s); iters.append(it)
        else:
            Jopt = np.linalg.solve(H, -q)
        # gerçek sistem: planın ilk ZMP komutunu uygula; ZMP fiziksel olarak ayak tabanında kalmak zorunda
        s_next = Ad @ s + Bd * Jopt[0]
        p_cmd = C @ s_next; foot = foot_at(t + dt / 2)
        x, v = s[0], s[1]
        for _ in range(50):                                    # 1 ms adımlarla gerçek LIPM
            p = np.clip(p_cmd, foot + back, foot + front)
            acc = w2 * (x - p); v += acc * 1e-3; x += v * 1e-3
        t += dt
        if abs(t - push_t) < dt / 2: v += push_dv
        s = np.array([x, v, w2 * (x - p)])
        log.append((t, x, p_cmd, p, foot))
    return np.array(log), iters

for dv in [0.0, 0.20, 0.30, 0.35]:
    for cons in [False, True]:
        lg, it = run(cons, dv)
        out = np.maximum(lg[:, 2] - (lg[:, 4] + front), (lg[:, 4] + back) - lg[:, 2]).max()
        fell = abs(lg[-1, 1] - foot_at(3.0)) > 0.05
        if fell:
            print(f"itme {dv:.2f} m/s, {'kısıtlı MPC (ADMM)' if cons else 'kısıtsız (kapalı form)'}: DÜŞTÜ"); continue
        print(f"itme {dv:.2f} m/s, {'kısıtlı MPC (ADMM)' if cons else 'kısıtsız (kapalı form)'}: "
              f"ZMP komutu ayak dışına max {max(out, 0) * 100:5.2f} cm, son CoM - son ayak = {100 * (lg[-1, 1] - foot_at(3.0)):5.2f} cm"
              + (f"  | ADMM iter. ort. {np.mean(it):.0f}, max {max(it)}" + (" (bazı adımlarda QP olurlu değil)" if max(it) >= 4000 else "") if cons else ""))

# Doğrulama: tek bir QP'yi SciPy ile çöz, karşılaştır
from scipy.optimize import minimize
s0 = np.array([0.02, 0.15, 0.0]); tk = dt * np.arange(1, N + 1); f = np.array([foot_at(tt - dt / 2) for tt in tk])
q = g_z * Pu.T @ (Px @ s0 - f - 0.01); lo, hi = f + back - Px @ s0, f + front - Px @ s0
xa, it = ADMM(H, Pu).solve(q, lo, hi, eps=1e-7, max_iter=20000)
cons = [{"type": "ineq", "fun": lambda J: Pu @ J - lo, "jac": lambda J: Pu}, {"type": "ineq", "fun": lambda J: hi - Pu @ J, "jac": lambda J: -Pu}]
rs = minimize(lambda J: 0.5 * J @ H @ J + q @ J, np.zeros(N), jac=lambda J: H @ J + q, constraints=cons, method="SLSQP", options={"maxiter": 500, "ftol": 1e-14})
obj = lambda J: 0.5 * J @ H @ J + q @ J
print(f"doğrulama: ADMM amaç {obj(xa):.8f} ({it} iterasyon), SciPy SLSQP amaç {obj(rs.x):.8f}; ZMP farkı max {np.abs(Pu @ (xa - rs.x)).max() * 1000:.3f} mm")
print(f"QP boyutu: {N} değişken, {N} kısıt; H koşul sayısı {np.linalg.cond(H):.1e}")
```

Çıktı:

```
itme 0.00 m/s, kısıtsız (kapalı form): ZMP komutu ayak dışına max  0.00 cm, son CoM - son ayak =  0.85 cm
itme 0.00 m/s, kısıtlı MPC (ADMM): ZMP komutu ayak dışına max  0.00 cm, son CoM - son ayak =  0.85 cm  | ADMM iter.
ort. 19, max 30
itme 0.20 m/s, kısıtsız (kapalı form): ZMP komutu ayak dışına max  0.00 cm, son CoM - son ayak =  0.85 cm
itme 0.20 m/s, kısıtlı MPC (ADMM): ZMP komutu ayak dışına max  0.00 cm, son CoM - son ayak =  0.85 cm  | ADMM iter.
ort. 21, max 100
itme 0.30 m/s, kısıtsız (kapalı form): ZMP komutu ayak dışına max  3.97 cm, son CoM - son ayak =  0.87 cm
itme 0.30 m/s, kısıtlı MPC (ADMM): ZMP komutu ayak dışına max  1.77 cm, son CoM - son ayak =  0.89 cm  | ADMM iter.
ort. 665, max 4000 (bazı adımlarda QP olurlu değil)
itme 0.35 m/s, kısıtsız (kapalı form): DÜŞTÜ
itme 0.35 m/s, kısıtlı MPC (ADMM): DÜŞTÜ
doğrulama: ADMM amaç -0.01009296 (410 iterasyon), SciPy SLSQP amaç -0.01009296; ZMP farkı max 0.008 mm
QP boyutu: 32 değişken, 32 kısıt; H koşul sayısı 1.3e+05
```

C++: `QpAdmm` sınıfı ve MPC döngüsü

```cpp
// w3_admm.cpp — ADMM QP çözücü sınıfı (OSQP'nin çekirdek fikri) + LIPM ZMP MPC'si, Eigen ile
#include <Eigen/Dense>
#include <algorithm>
#include <chrono>
#include <cmath>
#include <cstdio>
using Eigen::MatrixXd;
using Eigen::VectorXd;

//   min ½ xᵀPx + qᵀx   s.t.  l ≤ A x ≤ u
class QpAdmm {
public:
    QpAdmm(const MatrixXd& P, const MatrixXd& A, double rho = 0.1, double sigma = 1e-6, double alpha =
1.6)
        : P_(P), A_(A), rho_(rho), sigma_(sigma), alpha_(alpha),
          llt_(P + sigma * MatrixXd::Identity(P.rows(), P.cols()) + rho * A.transpose() * A),   // bir
kez ayrıştır
          x_(VectorXd::Zero(P.rows())), z_(VectorXd::Zero(A.rows())), y_(VectorXd::Zero(A.rows())) {}

    int solve(const VectorXd& q, const VectorXd& l, const VectorXd& u, double eps = 1e-5, int max_iter
= 4000) {
        z_ = z_.cwiseMax(l).cwiseMin(u);                    // sıcak başlangıç, kısıt kümesine izdüşür
        int it = 1;
        for (; it <= max_iter; ++it) {
            const VectorXd xt = llt_.solve(sigma_ * x_ - q + A_.transpose() * (rho_ * z_ - y_));
            const VectorXd zt = A_ * xt;
            x_ = alpha_ * xt + (1 - alpha_) * x_;
            const VectorXd zr = alpha_ * zt + (1 - alpha_) * z_;
            z_ = (zr + y_ / rho_).cwiseMax(l).cwiseMin(u);
            y_ += rho_ * (zr - z_);
            if (it % 10 == 0) {
                const double rp = (A_ * x_ - z_).cwiseAbs().maxCoeff();
                const double rd = (P_ * x_ + q + A_.transpose() * y_).cwiseAbs().maxCoeff();
                if (rp < eps && rd < eps * 10) break;
            }
        }
        return std::min(it, max_iter);
    }
    const VectorXd& x() const { return x_; }

private:
    MatrixXd P_, A_;
    double rho_, sigma_, alpha_;
    Eigen::LLT<MatrixXd> llt_;
    VectorXd x_, z_, y_;
```

```cpp
};

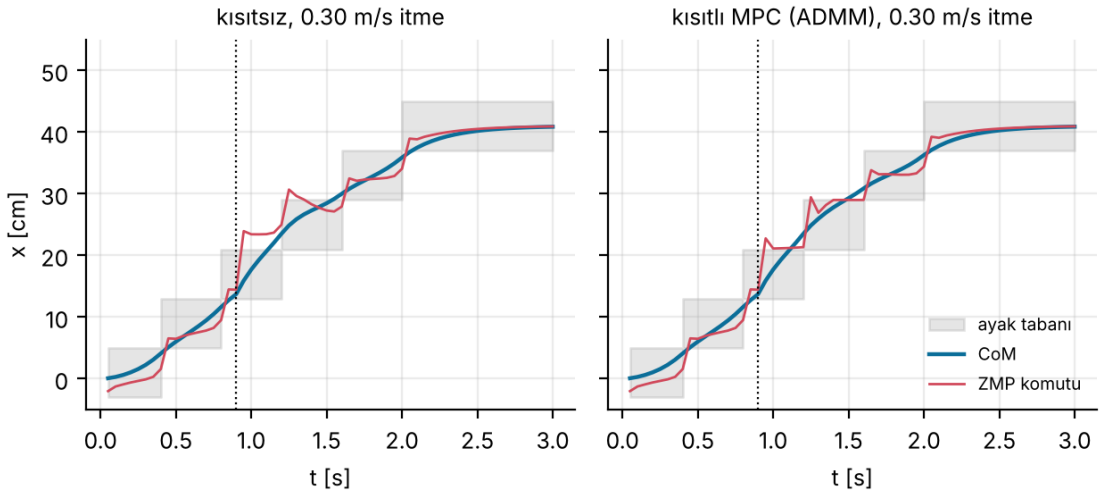
int main() {
    const double g = 9.81, zc = 0.32, w2 = g / zc, dt = 0.05;
    const int N = 32;
    Eigen::Matrix3d Ad; Ad << 1, dt, dt * dt / 2, 0, 1, dt, 0, 0, 1;
    const Eigen::Vector3d Bd(dt * dt * dt / 6, dt * dt / 2, dt);
    const Eigen::RowVector3d C(1, 0, -1 / w2);
    MatrixXd Px(N, 3), Pu = MatrixXd::Zero(N, N);
    Eigen::Matrix3d Ak = Eigen::Matrix3d::Identity();
    for (int i = 0; i < N; ++i) {
        Ak = Ad * Ak; Px.row(i) = C * Ak;
        Eigen::Matrix3d Ap = Eigen::Matrix3d::Identity();
        for (int j = i; j >= 0; --j) { Pu(i, j) = C * Ap * Bd; Ap = Ad * Ap; }   // Pu(i,j) = C Ad^(i-j) Bd
    }
    const double a_j = 1e-6, g_z = 1.0, T_step = 0.4, L_step = 0.08, back = -0.03, front = 0.05;
    const MatrixXd H = a_j * MatrixXd::Identity(N, N) + g_z * Pu.transpose() * Pu;
    auto foot_at = [&](double t) { return L_step * std::min(int(t / T_step), 5); };

    for (double dv : {0.0, 0.30}) {
        QpAdmm qp(H, Pu);
        Eigen::Vector3d s = Eigen::Vector3d::Zero();
        double t = 0, out_max = 0, x = 0, p = 0; int it_sum = 0, it_max = 0, steps = 0;
        auto t0 = std::chrono::steady_clock::now();
        while (t < 3.0 - 1e-9) {
            VectorXd f(N);
            for (int i = 0; i < N; ++i) f(i) = foot_at(t + dt * (i + 1) - dt / 2);
            const VectorXd q = g_z * Pu.transpose() * (Px * s - (f.array() + 0.01).matrix());
            const VectorXd base = Px * s;
            const int it = qp.solve(q, (f.array() + back).matrix() - base, (f.array() + front).matrix() - base);
            it_sum += it; it_max = std::max(it_max, it); ++steps;
            const double p_cmd = C * (Ad * s + Bd * qp.x()(0));
            const double foot = foot_at(t + dt / 2);
            out_max = std::max({out_max, p_cmd - (foot + front), (foot + back) - p_cmd});
            x = s(0); double v = s(1);
            for (int k = 0; k < 50; ++k) {                    // 1 ms adımlarla gerçek LIPM, ZMP ayakta kırpılı
                p = std::clamp(p_cmd, foot + back, foot + front);
                v += w2 * (x - p) * 1e-3; x += v * 1e-3;
            }
            t += dt;
            if (std::abs(t - 0.9) < dt / 2) v += dv;
            s << x, v, w2 * (x - p);
        }
        const double ms = std::chrono::duration<double, std::milli>(std::chrono::steady_clock::now() - t0).count();
        std::printf("itme %.2f m/s, kısıtlı MPC: ZMP komutu ayak dışına max %.2f cm, son CoM - son ayak = %.2f cm | ADMM iter. ort. %.0f, max %d | 60 MPC adımı toplam %.1f ms\n",
                    dv, std::max(out_max, 0.0) * 100, (x - foot_at(3.0)) * 100, double(it_sum) / steps, it_max, ms);
    }
    return 0;
}
```

Çıktı:

```
itme 0.00 m/s, kısıtlı MPC: ZMP komutu ayak dışına max 0.00 cm, son CoM - son ayak = 0.85 cm | ADMM iter. ort. 19, max 30 | 60 MPC adımı toplam 1.9 ms
```

```
itme 0.30 m/s, kısıtlı MPC: ZMP komutu ayak dışına max 1.77 cm, son CoM - son ayak = 0.89 cm | ADMM iter. ort. 665, max 4000 | 60 MPC adımı toplam 63.8 ms
```



Şekil 3.1: 0.30 m/s itme. Sol: kısıtsız plan ZMP'yi ayak dışına istiyor (gerçekte kırpılıyor). Sağ: kısıtlı MPC.

C++ notu: üye başlatma sırası ve `Eigen::LLT`

`QpAdmm` kurucusunda `llt_(P + sigma*I + rho*A'A)` ayrışımı *başlatma listesinde* bir kez yapılıyor; `solve` sadece `llt_.solve(...)` çağırıyor. Üyeler sınıfta *bildirildikleri sırayla* başlatılır (listedeki sırayla değil): `llt_`, `P_` ve `A_`'dan sonra bildirildiği için güvenli. `cwiseMax(l).cwiseMin(u)` vektörel kırpma.

Bölüm 4

Merkezcil Dinamik ve Tüm-Gövde Kontrolü

4.1 Merkezcil dinamik

Robot ne kadar eklemli olursa olsun, dış kuvvetler sadece temaslardan gelir. Toplam doğrusal ve açısal momentumun değişimi (Orin ve Goswami):

$$m\ddot{\mathbf{c}} = \sum_i \mathbf{f}_i + m\mathbf{g}, \qquad \dot{\mathbf{L}} = \sum_i (\mathbf{p}_i - \mathbf{c}) \times \mathbf{f}_i.$$

LIPM bunun $\dot{\mathbf{L}} = 0$, $\ddot{c}_z = 0$ özel hâlidir. **Sürtünme konisi:** kaymamak için $\|\mathbf{f}_t\| \le \mu f_n$, $f_n \ge 0$. Düzlemde doğrusal: $f_z \ge 0$, $|f_x| \le \mu f_z$. 3B'de koni bir piramitle yaklaştırılır.

4.2 QP tabanlı tüm-gövde kontrolü (WBC)

Modern humanoid kontrolcülerinin çekirdeği, her kontrol adımında (500–1000 Hz) çözülen küçük bir QP'dir. Tam hâli: değişkenler eklem ivmeleri $\ddot{\mathbf{q}}$, temas kuvvetleri $\mathbf{f}$, torklar $\boldsymbol{\tau}$; eşitlik kısıtı yüzen tabanlı dinamik $\mathbf{M}\ddot{\mathbf{q}} + \mathbf{h} = \mathbf{S}^T\boldsymbol{\tau} + \mathbf{J}_c^T\mathbf{f}$; temas ayakları kaymasın: $\mathbf{J}_c\ddot{\mathbf{q}} + \dot{\mathbf{J}}_c\dot{\mathbf{q}} = 0$; sürtünme konileri, tork sınırları; maliyet, görev ivmelerinin (CoM, gövde yönelimi, salınım ayağı) izlenmesi. Bu cildin kodu çekirdek alt problemi çözüyor: **istenen merkezcil wrench'i, koni içinde kalan temas kuvvetlerine dağıtmak**, sonra her bacakta $\boldsymbol{\tau} = -\mathbf{J}^T\mathbf{f}$.

Sayılar: çift destek, ayaklar ± 6 cm, her ayakta topuk (-3 cm) ve burun ($+5$ cm)

durum	dikey yükler [N] (arka topuk, burun, ön topuk, burun)	yorum
CoM ortada	7.31, 6.37, 5.90, 4.95	toplam $24.5 = mg$
CoM ön ayak üstünde	0.24, 4.95, 7.31, 12.02	arka ayak neredeyse boş
ileri 3 m/s^2 ivme, $\mu = 0.8$	24.6, 0, 0, 0	ZMP $= -9.8$ cm: topuk sınırında, $F_x = 7.2$ N
aynısı, $\mu = 0.2$	17.4, 6.5, 1.1, 0	F_x en fazla 4.9 N: istenen 7.5 N'nin 2.5 N'si karşılanamıyor
CoM $x = 14$ cm (destek dışı)	0, 0, 0, 24.4	tüm yük burunda, wrench hatası var: **devriliyor**

QP imkânsızı istemiyor: konileri koruyup wrench hatasını en aza indiriyor. Bu "hata" üst seviyeye (MPC, adım planlayıcı) giden bir sinyal olmalı.

SB-1 için önemli bir tasarım sonucu

Tek ayak üstünde, tüm ağırlık (24.5 N) ayak kalçanın tam altındayken diz torku:

kalça yüksekliği	0.30 m	0.34 m	0.37 m	0.39 m
diz bükümü	82.8°	63.6°	44.7°	25.7°
diz torku	**3.24**	2.58	1.86	1.09 N·m

Elle: diz, ayaktan yatayda $0.2 \sin 41.4^\circ = 0.132$ m uzakta; $24.5 \cdot 0.132 = 3.24$ N·m. MG996R'nin durma torku 6 V'ta ≈ 1.08 N·m (datasheet) ve sürekli çalışmada bunun ancak bir kısmı kullanılabilir. Sonuç: **2.5 kg'lık SB-1, MG996R dizlerle çömelerek yürüyemez.** Seçenekler: (1) kütleyi 1–1.2 kg'a indirmek, (2) dizde daha güçlü servo (25–35 kgf·cm sınıfı), (3) dizi neredeyse düz tutan bir yürüyüş (ama o zaman LIPM'in sabit yükseklik varsayımı zorlaşır), (4) dize paralel bir yay (ağırlığın bir kısmını pasif taşır). Cilt 0'daki SB-1 tanımını bu bilgiyle güncelle; bu tam da "hesapla, sonra yap" mühendisliğidir.

Python

```python
import numpy as np
from qp_admm import ADMM

# Tüm-gövde kontrolünün (WBC) kalbi: istenen gövde hareketini, SÜRTÜNME KONİSİ içinde kalan temas kuvvetlerine ve
# oradan eklem torklarına çevirmek. Düzlemsel (sagittal) SB-1, çift destek, her ayakta topuk + burun temas noktası.
m, g, zc = 2.5, 9.81, 0.32
def contacts(x_back, x_front):                # 4 temas noktası: (arka ayak topuk, burun, ön ayak topuk, burun)
    return np.array([x_back - 0.03, x_back + 0.05, x_front - 0.03, x_front + 0.05])

def distribute(c, cdd, Ldot, pts, mu, w_lin=1.0, w_ang=10.0, reg=1e-4):
    """Değişkenler f = [fx1, fz1, ..., fx4, fz4]. Merkezcil (centroidal) dinamik:
         Σ f = m (c̈ - g)        Σ (p_i - c) × f_i = L̇   (düzlemde: (px-cx) fz - (pz-cz) fx, pz = 0)"""
    n = len(pts); Aw = np.zeros((3, 2 * n))
    for i, px in enumerate(pts):
        Aw[0, 2 * i] = 1; Aw[1, 2 * i + 1] = 1
        Aw[2, 2 * i] = -(0 - c[1]); Aw[2, 2 * i + 1] = (px - c[0])
    wdes = np.array([m * cdd[0], m * (cdd[1] + g), Ldot])
    W = np.diag([w_lin, w_lin, w_ang])
    P = Aw.T @ W @ Aw + reg * np.eye(2 * n); q = -Aw.T @ W @ wdes
    # sürtünme konisi (doğrusal): fz ≥ 0,  fx - μ fz ≤ 0,  -fx - μ fz ≤ 0  (her temas için 3 satır)
    A = np.zeros((3 * n, 2 * n)); lo = np.zeros(3 * n); hi = np.zeros(3 * n); BIG = 1e6
    for i in range(n):
        A[3 * i, 2 * i + 1] = 1;                          lo[3 * i], hi[3 * i] = 0, BIG
        A[3 * i + 1, 2 * i] = 1; A[3 * i + 1, 2 * i + 1] = -mu; lo[3 * i + 1], hi[3 * i + 1] = -BIG, 0
        A[3 * i + 2, 2 * i] = 1; A[3 * i + 2, 2 * i + 1] = mu;  lo[3 * i + 2], hi[3 * i + 2] = 0, BIG
    f, it = ADMM(P, A, rho=1.0).solve(q, lo, hi, eps=1e-6, max_iter=20000)
    return f.reshape(-1, 2), Aw @ f - wdes, it

pts = contacts(-0.06, 0.06)
cases = [("ayakta, CoM ortada",            np.array([0.00, zc]), np.array([0, 0]),   0.0, 0.8),
         ("CoM ön ayağın üstünde",         np.array([0.06, zc]), np.array([0, 0]),   0.0, 0.8),
         ("itmeye karşı ileri 3 m/s² ivme", np.array([0.00, zc]), np.array([3.0, 0]), 0.0, 0.8),
         ("aynısı, kaygan zemin μ =0.2",    np.array([0.00, zc]), np.array([3.0, 0]), 0.0, 0.2),
         ("CoM destek dışında (x =0.14)",   np.array([0.14, zc]), np.array([0, 0]),   0.0, 0.8)]
for name, c, cdd, Ld, mu in cases:
    F, err, it = distribute(c, cdd, Ld, pts, mu)
    load = F[:, 1]
    print(f"{name:32s} μ ={mu}: dikey yükler [N] = {np.round(load, 2)} | toplam Fx {F[:, 0].sum():5.2f} N | "
          f"wrench hatası {np.round(err, 2)} | ADMM {it} it.")
```

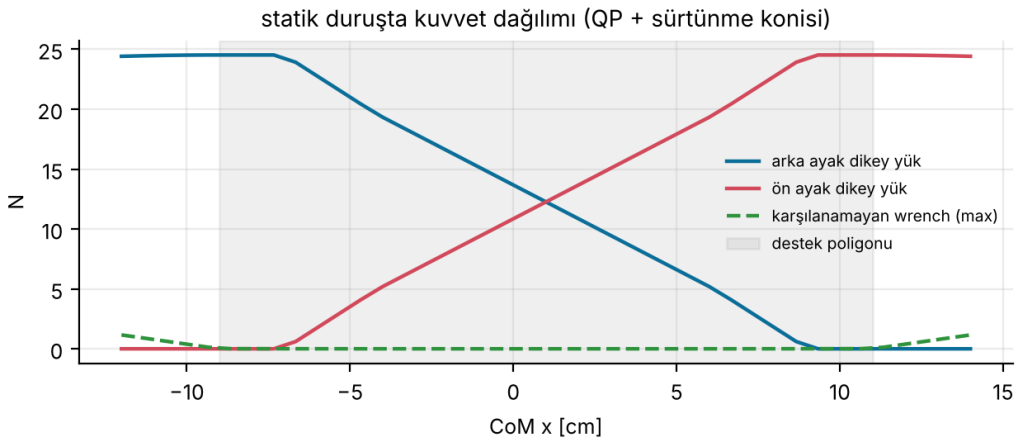
```python
print(f"Not: istenen Fx = m·c̈ = {m * 3.0:.1f} N; kaygan zeminde en fazla μ·m·g = {0.2 * m * g:.2f} N mümkün.")

# Temas kuvvetinden eklem torkuna:  τ = -Jᵀ f  (ayak yerde, kuvvet yerden ayağa; bacak gövdeyi taşıyor)
# Ön bacak, Cilt 1'deki 2-uzuvlu bacak (uyluk = baldır = 0.2 m), kalçaya göre ayak konumu
l1 = l2 = 0.20
def leg_ik(px, pz):                                  # kalçaya göre ayak (aşağı = -z), diz geriye bükük
    d = np.hypot(px, pz); q2 = np.arccos((d**2 - l1**2 - l2**2) / (2 * l1 * l2))
    q1 = np.arctan2(px, -pz) - np.arctan2(l2 * np.sin(q2), l1 + l2 * np.cos(q2))
    return q1, q2
def leg_jac(q1, q2):                                 # ayak konumu (x, z) = (l1 s1 + l2 s12, -l1 c1 - l2 c12)
    s1, c1, s12, c12 = np.sin(q1), np.cos(q1), np.sin(q1 + q2), np.cos(q1 + q2)
    return np.array([[l1 * c1 + l2 * c12, l2 * c12], [l1 * s1 + l2 * s12, l2 * s12]])
for name, c, cdd, Ld, mu in cases[:3]:
    F, _, _ = distribute(c, cdd, Ld, pts, mu)
    f_front = F[2] + F[3]                            # ön ayağın toplam kuvveti (yerden ayağa)
    hip = np.array([c[0], zc - 0.02])                # kalça CoM'un 2 cm altında (sadeleştirme)
    q1, q2 = leg_ik(0.06 - hip[0], 0 - hip[1])
    tau = -leg_jac(q1, q2).T @ f_front
    print(f"{name:32s}: ön bacak q = ({np.degrees(q1):5.1f}°, {np.degrees(q2):5.1f}°), "
          f"τ_kalça = {tau[0]:6.3f}, τ_diz = {tau[1]:6.3f} N·m   (MG996R durma torku ≈ 1.08 N·m)")

# Tasarım sorusu: tek ayak üstünde (tüm ağırlık, ayak kalçanın tam altında) diz torku kalça yüksekliğine nasıl bağlı?
for h in [0.30, 0.34, 0.37, 0.39]:
    q1, q2 = leg_ik(0.0, -h)
    tau = -leg_jac(q1, q2).T @ np.array([0.0, m * g])
    print(f"kalça yüksekliği {h:.2f} m (diz {np.degrees(q2):5.1f}° büküк): τ_kalça {tau[0]:6.3f}, τ_diz {tau[1]:6.3f} N·m")
```

Çıktı:

```
ayakta, CoM ortada              μ =0.8: dikey yükler [N] = [7.31 6.37 5.9  4.95] | toplam Fx -0.00 N | wrench hatası [-0. -0.  0.] | ADMM 140 it.
CoM ön ayağın üstünde           μ =0.8: dikey yükler [N] = [ 0.24  4.95  7.31 12.02] | toplam Fx  0.00 N | wrench hatası [ 0. -0. -0.] | ADMM 160 it.
itmeye karşı ileri 3 m/s² ivme  μ =0.8: dikey yükler [N] = [24.6  0.   0.   0. ] | toplam Fx  7.21 N | wrench hatası [-0.29  0.08  0.09] | ADMM 250 it.
aynısı, kaygan zemin μ =0.2     μ =0.2: dikey yükler [N] = [17.41  6.53  1.08  0.  ] | toplam Fx  5.00 N | wrench hatası [-2.5  0.5  0. ] | ADMM 120 it.
CoM destek dışında (x =0.14)    μ =0.8: dikey yükler [N] = [ 0.    0.    0.   24.41] | toplam Fx  1.16 N | wrench hatası [ 1.16 -0.11 -0.36] | ADMM 120 it.
Not: istenen Fx = m·c̈ = 7.5 N; kaygan zeminde en fazla μ·m·g = 4.91 N mümkün.
ayakta, CoM ortada              : ön bacak q = (-28.8°,  80.2°), τ_kalça = -0.651, τ_diz = -1.696 N·m   (MG996R durma torku ≈ 1.08 N·m)
CoM ön ayağın üstünde           : ön bacak q = (-41.4°,  82.8°), τ_kalça = -0.000, τ_diz = -2.557 N·m   (MG996R durma torku ≈ 1.08 N·m)
itmeye karşı ileri 3 m/s² ivme  : ön bacak q = (-28.8°,  80.2°), τ_kalça =  0.000, τ_diz =  0.000 N·m   (MG996R durma torku ≈ 1.08 N·m)
kalça yüksekliği 0.30 m (diz  82.8° büküк): τ_kalça  0.000, τ_diz -3.244 N·m
kalça yüksekliği 0.34 m (diz  63.6° büküк): τ_kalça  0.000, τ_diz -2.584 N·m
kalça yüksekliği 0.37 m (diz  44.7° büküк): τ_kalça  0.000, τ_diz -1.864 N·m
kalça yüksekliği 0.39 m (diz  25.7° büküк): τ_kalça -0.000, τ_diz -1.090 N·m
```



Şekil 4.1: Statik duruşta CoM konumuna göre ayak yükleri. Destek poligonunun dışında QP isteneni karşılayamıyor.

Bölüm 5

Öğrenme Tabanlı Yürüme

Son yıllarda bacaklı robotlarda en çarpıcı sonuçlar (ETH'nin ANYmal'ı, Berkeley'nin Cassie ve humanoid çalışmaları, Unitree ve Booster robotlarının gösterileri) büyük ölçüde **pekiştirmeli öğrenme** (RL) ile simülasyonda eğitilip gerçek robota aktarılan politikalarla geldi. Bu, Bölüm 1–4'ü gereksiz kılmıyor: ödül tasarımı, gözlem seçimi, güvenlik ve sorun ayıklama hep bu modellerin sezgisine dayanıyor; birçok güncel sistem de ikisini birleştiriyor.

5.1 RL ile yürüme: tarifin parçaları

- **Politika** $\pi_\theta(\mathbf{a} \mid \mathbf{o})$: küçük bir sinir ağı (ör. 2–3 katman, 256–512 nöron), 50 Hz civarı.
- **Gözlem o**: gövde açısal hızı ve yerçekimi yönü (IMU'dan; Cilt 5 Bölüm 7), eklem açıları ve hızları, önceki eylem, komut (istenen hız), çoğu zaman bir faz saati. Gövde doğrusal hızı ya kestirilir ya da hiç verilmez.
- **Eylem a**: genelde eklem hedef açıları; alt seviyede yüksek hızlı PD (Cilt 4A) bunu torka çevirir. Bu yüzden PD kazançları ve aktüatör modeli politikayla birlikte "öğrenilmiş" bir sistemin parçasıdır.
- **Ödül**: hız komutunu izleme (ana terim) + cezalar (tork, eklem hızı, eylem değişimi, ayak kayması, gövde yönelimi, düşme) + bazen ayak havada kalma süresi. Ödül ağırlıklarıyla oynamak işin büyük kısmı.
- **Algoritma**: çoğunlukla PPO (on-policy), binlerce paralel ortamla GPU'da (Isaac Gym/Lab, MuJoCo MJX / Playground). Dakikalar–saatler.
- **Sim-to-real**: alan rastgeleleştirme (*domain randomization*: kütle, sürtünme, motor gücü, gecikme, gürültü, itmeler), aktüatör modeli (gerçek motordan toplanan veriyle eğitilmiş bir ağ veya Cilt 3'teki gibi fiziksel model), öğretmen–öğrenci eğitimi (öğretmen simülasyonun gizli bilgilerini görür, öğrenci sadece gerçek sensörleri).

5.2 Taklit ve difüzyon politikaları

Taklit öğrenme: insan hareket yakalama verisini (retargeting ile robota uyarlanmış) izleyen ödüller; humanoidlerin "insansı" yürüyüş ve dans gösterileri genelde buradan. **Difüzyon politikaları** (Chi ve ark., Diffusion Policy): eylem dizisini bir difüzyon modeliyle üretir; özellikle manipülasyonda, gösterimlerden öğrenmede güçlü. Yürüyüşte RL hâlâ baskın; ama "genel amaçlı" robot modelleri (görme-dil-eylem, VLA) üst seviye kararları alıp alt seviyeye (RL veya MPC+WBC) komut veriyor.

5.3 Model tabanlı mı, öğrenme mi?

	MPC + WBC (Bölüm 3–4)	RL politikası
Gereken	doğru model, hızlı çözücü	simülatör, GPU, ödül mühendisliği
Güçlü yanı	yorumlanabilir, kısıtları açıkça korur, yeni göreve hızlı uyarlanır	engebeli zemin, temas belirsizliği, hızlı ve "refleksif" tepkiler
Zayıf yanı	model hataları, temas zamanlaması varsayımları	veri/hesap açlığı, garantisiz, hata ayıklaması zor
Hesap (robotta)	her adımda QP	tek bir ileri geçiş (çok ucuz)

5.4 SB-1 için bir RL planı (simülasyonda)

1. SB-1 MJCF modeli (Cilt 2), servo modeli olarak Cilt 3'teki PD + doyma + 20 ms gecikme.
2. Gözlem: IMU yerçekimi yönü ve açısal hız, 10 eklem açısı/hızı, önceki eylem, komut hızı. Eylem: 10 eklem hedef açısı (nominal duruşa göre ± 0.5 rad).
3. Ödül: hız takibi + tork cezası (Bölüm 4'teki diz torku sorunu RL'de de var: politika düz dizli yürümeyi *kendisi* bulabilir!).
4. Rastgeleleştirme: kütle $\pm 20\%$, sürtünme 0.4–1.0, servo gecikmesi 10–30 ms, itmeler.
5. Karşılaştırma ölçütleri: Bölüm 2'deki itme toleransı, enerji, hız takibi. MPC+WBC ile aynı testleri uygula.

5.5 Okuma listesi

- Hwangbo ve ark., "Learning agile and dynamic motor skills for legged robots", Science Robotics 2019 (aktüatör ağı).
- Lee ve ark., "Learning quadrupedal locomotion over challenging terrain", Science Robotics 2020 (öğretmen–öğrenci).
- Rudin ve ark., "Learning to walk in minutes using massively parallel deep RL", CoRL 2021 (`legged_gym`).
- Radosavovic ve ark., "Real-world humanoid locomotion with reinforcement learning", Science Robotics 2024.
- Chi ve ark., "Diffusion Policy", RSS 2023.
- Tedrake, *Underactuated Robotics*, "Reinforcement learning" ve "Imitation learning" bölümleri (Cilt 4B Bölüm 20–21).

Bölüm 6

SB-1 Yazılım Mimarisi ve Proje Yol Haritası

6.1 Katmanlar ve hızlar

Katman	Hız	Girdi → çıktı	Cilt
Eklem servo/PD	1 kHz (servoda)	hedef açı → tork/PWM	3, 4A
Durum kestirimi	400–1000 Hz	IMU, enkoder, temas → gövde hızı, yönelim	5 (Bölüm 7)
WBC (QP)	200–1000 Hz	görev ivmeleri → tork/hedef açı	6 (Bölüm 4)
MPC / DCM	20–100 Hz	adım planı, durum → CoM/ZMP yörüngesi	6 (Bölüm 1–3)
Adım planlayıcı	adım başına	hız komutu, harita → ayak yerleri	6 (Bölüm 2), 5
Algı / SLAM	10–30 Hz	kamera/lidar → harita, poz	5
Görev	olay	kullanıcı → hız komutu	–

Her katman, altındakinin kapalı çevrim bant genişliğinden yavaş çalışır (Cilt 4A Bölüm 3'teki 10 kat kuralı). Hızlı katmanlar C++ ve gerçek zamanlı iş parçacıklarında (Cilt 2), yavaşlar Python veya ayrı ROS 2 düğümlerinde olabilir.

6.2 ROS 2 düğüm taslağı

- `/sb1/hardware`: `ros2_control` donanım arayüzü (Arduino seri bağlantı); `joint_states`, `imu`.
- `/sb1/estimator`: Cilt 5'teki KF; `odom` yayınlar.
- `/sb1/walking_controller`: DCM/MPC + WBC tek bir `ros2_control` denetleyicisi içinde (aynı iş parçacığı, mesaj gecikmesi yok).
- `/sb1/teleop`: `cmd_vel`.

Hızlı döngüyü ROS mesajlarıyla bölmemek önemli: 1 kHz'lik bir döngüde düğümler arası mesajlaşma gecikmesi ve titremesi kontrolü bozar.

6.3 Proje basamakları (Cilt 0'daki P1–P8 ile uyumlu)

1. **Masaüstü bacak** (2 MG996R + Arduino): Cilt 3'teki servo karakterizasyonu, Cilt 1 IK ile ayak yörüngesi.
2. **Simülasyonda SB-1 MJCF**: kütle–atalet hesapları, eklem sınırları, ayak temas geometrisi. Bölüm 4'teki diz torku sonucuna göre kütle/servo seçimi.

3. **Simülasyonda statik denge ve çömelme**: kuvvet dağılımı QP + PD.
4. **DCM yürüyüşü (simülasyon)**: önce sagittal, sonra yan salınım; Bölüm 1 kodu C++'a, döngü 1 kHz.
5. **İtme testleri**: Bölüm 2 tablosunu simülasyonda ölç; adım ayarlı MPC.
6. **RL politikası** (Bölüm 5) ve karşılaştırma.
7. **Donanım** (bütçe olunca): yalnızca simülasyonda sağlam çalışan denetleyicileri, düşük kazançla ve askıda test et.

Lisansüstü başvurusu için

Bu basamakların her biri bir GitHub deposu + kısa bir teknik rapor + video olabilir. Özellikle "hesapla, simülasyonda doğrula, gerçek donanımın sınırını ölç" zinciri (ör. Bölüm 4'teki diz torku analizi ve onu doğrulayan bir deney) hocaların görmek istediği mühendislik olgunluğunu gösterir.

Bölüm 7

Sorular

Kâğıt, kalem, hesap makinesi. SB-1: $m = 2.5$ kg, $z_c = 0.32$ m, $g = 9.81$. **(K)**: kod okuma.

W1 — LIPM ve DCM

W1.1 ω ve zaman sabiti? $z_c = 0.5$ m olan daha uzun bir robotta?

W1.2 ZMP ayak bileğinde ($p = 0$), $x_0 = 2$ cm, $\dot{x}_0 = 0$: $t = 0.4$ s'de x? ($\cosh 2.215 \approx 4.63$)

W1.3 $x = 1$ cm, $\dot{x} = 0.1$ m/s: DCM nerede? ZMP DCM'ye konursa CoM nerede durur?

W1.4 $\dot{\xi} = \omega(\xi - p)$ ve $\dot{x} = -\omega(x - \xi)$ ifadelerini $\ddot{x} = \omega^2(x - p)$'den türet.

W1.5 Ayaklar 32 ve 40 cm, adım sonunda (son ayakta) DCM 40 cm'de. $T = 0.4$ s ($e^{-\omega T} = 0.109$). 32 cm'deki ayağın adım sonu DCM'si? Bir önceki (24 cm) adımınki?

W1.6 CoM ivmesi $\ddot{x} = 3$ m/s^2, $x = 0$: ZMP nerede? SB-1'in topuğu bileğin 3 cm gerisinde ve iki ayak ± 6 cm'de ise bu mümkün mü?

W2 — Yakalanabilirlik

W2.1 Ayak önü 5 cm: ayak bileği stratejisiyle kurtarılabilen en büyük itme hızı?

W2.2 Salınım 0.3 s ($e^{-\omega T_s} = 0.19$), erim 20 cm: 1-adım sınırı?

W2.3 Neden "hızlı adım uzun bacaktan daha değerli"? Tabloya bakarak açıkla.

W3 — MPC ve QP

W3.1 Ufuk 1.6 s, $\Delta t = 50$ ms: QP'nin değişken sayısı? ZMP kısıtı sayısı?

W3.2 ADMM'de $\mathbf{z}$ adımı neden sadece bir `clip`? Matris ayrışımı neden döngü dışında?

W3.3 MPC çözücü bir adımda yakınsamazsa (olurlu değil) bu robot için ne anlama gelir; ne yapılmalı?

W3.4 (K) `run` fonksiyonunda `p = np.clip(p_cmd, ...)` satırı neyi modelliyor? Kaldırılsaydı kısıtsız MPC'nin sonuçları neden yanıltıcı olurdu?

W4 — Kuvvetler ve torklar

W4.1 Kaygan zemin $\mu = 0.2$: SB-1'in alabileceği en büyük yatay ivme?

W4.2 Tek ayakta, kalça 0.30 m yükseklikte, ayak kalçanın tam altında, uyluk = baldır = 0.2 m. Diz torkunu elle hesapla.

W4.3 Aynı robotun kütlesi 1.2 kg olsaydı ve dizde MG996R'nin durma torkunun yarısını (0.54 N·m) sürekli kullanabilseydik, en fazla ne kadar çömelebilirdi (diz ile ayak arasındaki yatay mesafe)?

W4.4 Sürtünme konisi kısıtlarını $f_z \geq 0$, $|f_x| \leq \mu f_z$ doğrusal eşitsizlik satırları olarak yaz.

W5–6 — Öğrenme ve mimari

W5.1 RL politikası için SB-1'de 5 gözlem ve 3 ceza terimi say. Neden gövde doğrusal hızı genelde gözleme konmaz?

W5.2 Alan rastgeleleştirmesinde hangi 4 parametre SB-1 için en önemli olurdu, neden?

W6.1 WBC neden MPC'den daha hızlı çalışır? MPC 50 Hz ise WBC'nin en az kaç Hz olması mantıklı?

Bölüm 8

Cevaplar

W1

W1.1 $\omega = \sqrt{9.81/0.32} = 5.54$ 1/s, $1/\omega = 181$ ms. $z_c = 0.5$: $\omega = 4.43$, 226 ms: uzun robot daha "yavaş düşer", denge daha kolay.

W1.2 $x = 0.02\cosh(5.537 \cdot 0.4) = 0.02 \cdot 4.63 = 9.3$ cm.

W1.3 $\xi = 0.01 + 0.1/5.537 = 0.0281$ m $= 2.81$ cm. CoM 2.81 cm'de durur.

W1.4 $\dot{\xi} = \dot{x} + \ddot{x}/\omega = \dot{x} + \omega(x - p) = \omega(x + \dot{x}/\omega - p) = \omega(\xi - p)$. $\dot{x} = \omega(\xi - x) = -\omega(x - \xi)$.

W1.5 $\xi = 32 + 0.109 \cdot (40 - 32) = 32.87$ cm. Önceki: $24 + 0.109 \cdot (32.87 - 24) = 24.97$ cm.

W1.6 $p = x - \ddot{x}/\omega^2 = -3/30.66 = -9.8$ cm. Arka ayağın topuğu $-6 - 3 = -9$ cm: ZMP desteğin 0.8 cm dışında, tam olarak mümkün değil (Bölüm 4'te QP'nin küçük wrench hatası buydu).

W2

W2.1 $5.537 \cdot 0.05 = 0.277$ m/s.

W2.2 $\omega(a + re^{-\omega T_s}) = 5.537(0.05 + 0.2 \cdot 0.19) = 0.487$ m/s.

W2.3 Salınım boyunca DCM $e^{\omega T_s}$ ile kaçar; bacak erimi bu üstel çarpanla bölünerek etki eder. $T_s = 0.6$ s'de $e^{-\omega T_s} = 0.036$: 20 cm erim sadece 0.7 cm'lik ek DCM toleransına dönüşüyor. $T_s = 0.15$ s'de 0.436: 8.7 cm.

W3

W3.1 $1.6/0.05 = 32$ değişken (her adım bir jerk), 32 çift taraflı kısıt.

W3.2 Kısıt kümesi bir kutu ($\mathbf{l} \le \mathbf{z} \le \mathbf{u}$); bir kutuya izdüşüm her bileşeni ayrı ayrı kırpmaktır. Matris $\mathbf{P} + \sigma\mathbf{I} + \rho\mathbf{A}^T\mathbf{A}$ iterasyonlar boyunca değişmez (ρ sabitken), Cholesky bir kez yapılır.

W3.3 Mevcut adım planıyla ZMP'yi destek içinde tutarak durumu kurtaracak bir giriş dizisi yok: robot bu planla düşecek. Adım yerini/zamanını değiştir (Bölüm 2), açısal momentum kullan, ya da güvenli düşme davranışına geç.

W3.4 Fiziksel gerçeği: ZMP destek dışına çıkamaz. Kaldırılsaydı simülasyon ayak dışındaki ZMP'yi "uygulayabilir" sayardı ve kısıtsız MPC'nin hiç düşmediği, sahte iyi sonuçlar çıkardı.

W4

W4.1 $F_x \le \mu mg \Rightarrow a \le \mu g = 1.96$ m/s^2.

W4.2 Uyluk dikeyle α açısı: $\cos\alpha = 0.15/0.2 = 0.75$, $\alpha = 41.4^\circ$; diz yatayda $0.2\sin\alpha = 0.132$ m. $\tau = 24.5 \cdot 0.132 = 3.24$ N·m.

W4.3 $mg = 11.8$ N; $0.54/11.8 = 0.046$ m: diz ile ayak arası yatayda en fazla 4.6 cm. $\sin\alpha = 0.23$, $\alpha = 13.3^\circ$, kalça yüksekliği $2 \cdot 0.2\cos\alpha = 0.389$ m: neredeyse düz diz.

W4.4 $[0 \;\; 1]\mathbf{f} \geq 0$; $[1 \;\; -\mu]\mathbf{f} \leq 0$; $[1 \;\; \mu]\mathbf{f} \geq 0$.

W5–6

W5.1 Gözlem: yerçekimi yönü (IMU), gövde açısal hızı, eklem açıları, eklem hızları, önceki eylem, komut hızı, faz saati. Ceza: tork, eylem değişimi, ayak kayması, eklem hızı, gövde eğimi. Doğrusal hız doğrudan ölçülemez (Cilt 5: kestirilir); simülasyonda kusursuz verilip gerçekte kestirimle verilirse sim-to-real farkı oluşur.

W5.2 Servo gecikmesi (20 ms mertebesi, kararlılığı belirler), servo tork sınırı/gücü (Bölüm 4: sınırda), zemin sürtünmesi, gövde kütlesi ve CoM konumu (montaj, kablolar, pil).

W6.1 WBC temas kuvvetlerini ve torkları üretir; bunlar hızlı değişen bozuculara ve modelleme hatalarına anında tepki vermeli. MPC daha uzun vadeli planı yapar. 10 kat kuralı: en az 500 Hz.